COLLÈGE LASALLE — COURS 420-TT4-AS

Les Tendances Technologiques

Travail Pratique — React, premier contact

React + TypeScript : comprendre, configurer, coder, déployer

Votre première application React de A à Z — du projet Vite au déploiement

TypeScript · Vite · WebStorm · Déploiement Vercel ou Render · Durée 3 h

Règle d'or : vous tapez chaque ligne à la main et vous l'expliquez à voix haute. Tout code copié-collé sans explication = 0 au contrôle oral.

1. Présentation du TP

1.1 Pourquoi ce TP ?

Jusqu'ici vous avez programmé avec JavaScript et TypeScript « en node » : des scripts en console, des API Express, des bases de données. Ce TP vous fait passer à l'étape suivante, la plus utilisée dans l'industrie : la construction d'interfaces web modernes (front-end) avec React.

React n'est pas un nouveau langage. C'est une bibliothèque (library) JavaScript qui rend la création d'interfaces beaucoup plus simple, prévisible et maintenable. Vous l'écrirez en TypeScript, comme toujours : vos composants seront typés.

1.2 Ce que vous allez construire

Une application web « Galerie de tendances » qui affiche, filtre et recherche une liste de technologies (IA, Docker, Redis, TypeScript…) fournie par une API. En allant plus loin : un tableau de bord qui charge des données distantes (JSONPlaceholder ou une API que vous avez déjà écrite au TP3).

- Un projet React créé avec Vite (le standard actuel, plus rapide que Create React App).
- Des composants typés en TypeScript (.tsx) : en-tête, liste, carte, champ de recherche.
- Des interactions : état (useState), passages de données (props), chargement d'API (useEffect + fetch).
- Un déploiement en ligne sur Vercel (recommandé) ou Render.

1.3 Ce que vous saurez faire à la fin

- Expliquer ce qu'est React, comment il fonctionne et pourquoi on l'utilise.
- Créer un projet React + TypeScript de A à Z dans WebStorm.
- Comprendre la structure d'un projet Vite (répertoires src/, public/, fichiers clés).
- Écrire des composants réutilisables et leur passer des données (props).
- Gérer des données qui changent (state) et des événements (onClick, onChange).
- Charger et afficher des données d'une API en gérant les états chargement/erreur/vide.
- Déployer l'application sur Vercel ou Render et la rendre publique.

1.4 Comment ce TP fonctionne : de moins en moins d'aide

Comme au TP3, la première étape est très guidée (Étape A), la suivante légèrement moins (Étape B), puis vous devez réutiliser seules les mécaniques apprises. Les réponses sont structurées pour vous forcer à comprendre avant d'écrire :

Guide	Qué obtient l'étudiant
Étape A	Code presque complet : vous tapez, vous observez, vous expliquez.
Étape B	Demi-code : squelettes à compléter avec les mécaniques de A.
Étape C	Objectif + conseils : vous codez en appliquant ce que vous avez compris.

Étape D (bonus)	Autonome : tout le code vient de vous.

1.5 Règles du TP

- Remise : un lien GitHub public du projet + le lien du site déployé (Vercel ou Render).
- Le projet doit tourner en local avec npm run dev avant la remise.
- Vous devez être capable d'expliquer chaque fichier du projet (contrôle oral).
- Interdit : copier un projet entier d'un autre étudiant ou d'un générateur IA sans l'avoir compris. L'IA peut vous aider à comprendre, pas à produire le projet à votre place.
- Durée conseillée : 3 heures (4 h avec le bonus).

1.6 Plan et temps estimé

Étape	Contenu	Temps
A	Concepts React + configuration complète (WebStorm, Vite, TS)	60 min
B	Premiers composants, JSX, props et typage	50 min
C	État, événements et recherche (useState)	40 min
D	API : chargement, affichage, gestion des erreurs	40 min
E (bonus)	Formulaire + déploiement Vercel ou Render	30 min

2. Comprendre React avant de coder

2.1 Qu'est-ce que React ?

React est une bibliothèque JavaScript open-source créée par Meta (Facebook) en 2013 pour construire des interfaces utilisateur à partir de petits morceaux réutilisables appelés composants.

Au lieu de manipuler la page manuellement (« je prends cette div, je change son texte… »), vous déclarez ce que l'interface doit AFFICHER à partir de données. Quand les données changent, React s'occupe tout seul de mettre la page à jour.

2.2 À quoi ça ressemble ?

Une interface React, c'est une arborescence de composants. Chaque composant est une fonction qui retourne du JSX (une syntaxe qui ressemble à du HTML mais que l'on écrit en JavaScript/TypeScript) :

```
// App.tsx — un composant (une fonction qui retourne du JSX)
function App() {
  return (
    <div>
      <Header titre="TechTrends" />
      <ListeTechnologies />
    </div>
  );
}

export default App;
```

2.3 Le ving conceptuel : le DOM virtuel

Modifier le DOM (la page) directement est lent et source d'erreurs. React garde en mémoire une copie légère de l'interface appelée DOM virtuel. Lorsqu'une donnée change :

- React recalcule le nouveau DOM virtuel (rapide, en mémoire).
- Il compare avec l'ancien (différenciation, « diff »).
- Il ne modifie dans la vraie page QUE ce qui a réellement changé.

Vous n'avez donc jamais à écrire « document.getElementById(...).innerHTML = ... ». Vous dites ce que vous voulez afficher, React s'occupe du comment.

2.4 Le cycle de données : props et state

Concept	Rôle	Qui le modifie ?
props	Données reçues par un composant depuis son parent. En lecture seule pour l'enfant.	Le parent.
state	Données internes et changeantes du composant (recherche, sélection, formulaire…).	Le composant lui-même (via useState).
flux	Les données descendent du parent vers l'enfant (unidirectionnel).	—

Règle n°1 : on ne modifie JAMAIS une prop dans l'enfant. Règle n°2 : si une valeur change à l'écran, elle doit être dans un state (pas une variable normale).

2.5 Les hooks dont vous avez besoin pour ce TP

Hook	À quoi il sert	Exemple
useState	Créer une donnée d'état et la fonction pour la changer.	const [recherche, setRecherche] = useState("");
useEffect	Lancer un effet après le rendu (charger une API au démarrage…).	useEffect(() => { chercherDonnees(); }, []);

C'est tout pour un premier contact. Le reste (useContext, useReducer, useMemo…) viendra plus tard.

2.6 Vite, c'est quoi ce build tool ?

Aujourd'hui on n'écrit plus du JavaScript « brut » dans le navigateur : on utilise TypeScript, on découpe en fichiers, on installe des paquets npm. Un outil de build transforme tout ça en une application web prête à tourner. Vite est l'outil moderne de référence (créé par Evan You, l'auteur de Vue) : très rapide, config minimal, support TypeScript et React immédiat.

Concept	Explication
npm run dev	Serveur de développement : rechargement instantané dans le navigateur.
npm run build	Production : crée un dossier dist/ prêt à être hébergé.
npm run preview	Teste localement le build de production.
public/	Fichiers statiques copiés tels quels (images, favicon).
src/main.tsx	Point d'entrée : monte le composant App dans la page.

2.7 .tsx, c'est quoi ?

React utilise des composants qui mélangent « markup » (JSX) et logique. On utilise donc l'extension .tsx : un fichier TypeScript qui contient du JSX. Les fichiers .ts restent pour la logique sans affichage (types, fonctions pures, appels API).

À retenir avant de commencer

React = composants + JSX + props + state + effets.

Les données descendent (props). L'état re-monOnternal.taille.

On déclare ce qu'on affiche ; React met à jour le DOM.

Vite crée le projet ; npm run dev pour travailler ; npm run build puis déploiement pour la production.

3. Étape A — Configuration de A à Z (60 min)

A1. Vérifier Node.js

React tourne sur Node.js. Vérifiez votre version (comme au TP TypeScript) :

```
node --version
npm --version
```

Versions attendues : Node 22 ou plus récent, npm 10 ou plus récent.

A2. Créer le projet avec Vite dans WebStorm

WebStorm intègre Vite : pas besoin de terminal séparé. Attention : ne créez PAS le projet dans le dossier du cours ; créez un dossier dédié, ex. C:\Users\Vous\TechTrends-react.

```
npm create vite@latest techtrends-react -- --template react-ts
```

Cette commande pose une question (le nom, déjà fourni), puis crée le dossier techtrends-react avec le template React + TypeScript. Ensuite :

```
cd techtrends-react
npm install
```

Dans WebStorm

File → Open → sélectionner le dossier techtrends-react.

WebStorm détecte package.json et propose de synchroniser (« Trust Project », « Use npm scripts »).

Éditeur : File → Settings → Editor → Code Style → JavaScript : choisir typeface prettier si proposé.

Terminal intégré : bouton Terminal en bas (raccourci Alt+F12).

A3. Explorer la structure générée

Ouvrez le projet et repérez chaque fichier. Vous devez pouvoir montrer du doigt chacun :

Fichier/Dossier	Rôle
package.json	Liste des dépendances (react, react-dom, typescript, vite) et des scripts.
vite.config.ts	Configuration de Vite (plugins React, port, base…).
tsconfig.json	Configuration TypeScript (strict mode activé par défaut).
index.html	Page d'accueil du SPA : contient la balise .
public/	Asssets statiques (vite.svg, favicon…).
src/main.tsx	Point d'entrée : ReactDOM.createRoot(...).render(<App />).
src/App.tsx	Le composant racine de l'application.
src/App.css	Styles du composant App.
src/index.css	Styles globaux (reset, variable).

A4. Lancer le serveur de développement

```
npm run dev
```

WebStorm ouvrira l'adresse http://localhost:5173 dans le navigateur (le port peut varier). La page affiche le logo Vite et une interface React de démonstration. Ensuite :

- Ouvrez src/App.tsx, changez un texte, enregistrez : la page se met à jour instantanément (HMR).
- Ouvrez la console du navigateur (F12) : aucun message rouge attendu.

A5. Réussir : la page de démo tourne

Avant de continuer : npm run dev affiche « VITE v6+ ready » et la page répond. Si une erreur apparaît, notez-la dans la console du Terminal de WebStorm et corrigez avant de passer à la suite.

A6. Nettoyer l'exemple

Le template crée des fichiers de démonstration que vous allez remplacer. Supprimez le contenu des fichiers ci-dessous puis écrivez-les vous-même (leçon : on comprend mieux ce qu'on réécrit).

src/App.tsx — remplacez tout le contenu par :

```
function App() {
  return (
    <main>
      <h1>TechTrends — Galerie des tendances</h1>
    </main>
  );
}

export default App;
```

src/index.css — remplacez tout par :

```
body {
  margin: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f4f6fb;
  color: #1e293b;
}
```

Supprimez src/App.css et les fichiers src/assets/Logo.svg (ou gardez-les, mais ne les importez plus).

Vérifiez : la page affiche « TechTrends — Galerie des tendances » en titre.

A7. Checkpoint — questions pour vous-même

- Pourquoi y a-t-il une balise dans index.html ?
- Que fait exactement la ligne ReactDOM.createRoot(...).render(<App />) ?
- Dans quel fichier se trouvent les scripts npm (dev, build, preview) ?

4. Étape B — Composer une interface (50 min)

B1. Objectif

Créer une interface composée : une en-tête (Header), une liste (ListeTendances) et des cartes (CarteTechnologie). Vous passerez des données d'un composant parent à des enfants avec des props typées.

B2. Créer un composant Header

Créez le fichier src/components/Header.tsx :

```
interface HeaderProps {
  titre: string;
  phrase: string;
}

function Header({ titre, phrase }: HeaderProps) {
  return (
    <header>
      <h1>{titre}</h1>
      <p>{phrase}</p>
    </header>
  );
}

export default Header;
```

Décodez ce que vous venez d'écrire :

- HeaderProps : un type qui décrit les données attendues (props).
- function Header({ titre, phrase }: HeaderProps) : la fonction détruit les props pour en faire des variables.
- {titre} : en JSX, les accolades insèrent une expression TypeScript — ici la valeur de la prop.

B3. Créer un composant CarteTechnologie

Créez src/components/CarteTechnologie.tsx. Premier contact avec une interface de données :

```
export interface Technologie {
  id: number;
  nom: string;
  categorie: string;
  niveauPopularite: 1 | 2 | 3 | 4 | 5;
  couleur: string;
}

interface CarteProps {
  tech: Technologie;
}

function CarteTechnologie({ tech }: CarteProps) {
  return (
    <article style={{ borderLeft: '6px solid ' + tech.couleur, padding: 12 }}>
      <h3>{tech.nom}</h3>
      <p>{tech.categorie} · Popularité {tech.niveauPopularite}/5</p>
    </article>
  );
}

export default CarteTechnologie;
```

B4. Créer une liste avec des données typées

Créez src/data/technologies.ts — les données que l'application affichera (vous la remplacerez par une vraie API à l'étape D) :

```
import type { Technologie } from '../components/CarteTechnologie';

export const technologies: Technologie[] = [
  { id: 1, nom: 'React', categorie: 'Front-end', niveauPopularite: 5, couleur: '#61DAFB' },
  { id: 2, nom: 'TypeScript', categorie: 'Langage', niveauPopularite: 5, couleur: '#3178C6' },
  { id: 3, nom: 'Docker', categorie: 'Conteneurisation', niveauPopularite: 4, couleur: '#0DB7ED' },
  { id: 4, nom: 'Redis', categorie: 'Cache', niveauPopularite: 4, couleur: '#DC382D' },
  { id: 5, nom: 'PostgreSQL', categorie: 'Base de données', niveauPopularite: 4, couleur: '#336791'
},
  { id: 6, nom: 'Ollama', categorie: 'IA locale', niveauPopularite: 3, couleur: '#7C3AED' },
];
```

B5. Utiliser la liste dans App

Modifiez src/App.tsx pour afficher la liste :

```
import Header from './components/Header';
import CarteTechnologie from './components/CarteTechnologie';
import { technologies } from './data/technologies';

function App() {
  return (
    <main>
      <Header
        titre="TechTrends — Galerie des tendances"
        phrase="Les technologies qui font le cours 420-TT4"
      />
      <section>
        {technologies.map((tech) => (
          <CarteTechnologie key={tech.id} tech={tech} />
        ))}
      </section>
    </main>
  );
}

export default App;
```

Deux nouveautés à comprendre :

- technologies.map(...) : on transforme le tableau en liste de composants JSX.
- key={tech.id} : prop spéciale obligatoire pour que React identifie chaque élément de liste.

B6. Exercice guidé — typer un composant Video

Dans src/components/, créez Video.tsx : un composant qui reçoit un objet video { id, titre, dureeEnSecondes } et affiche le titre plus la durée en minutes (par ex. « 3 min 30 »). Commencez par déclarer l'interface. Astuce : Math.floor(dureeEnSecondes / 60) pour les minutes, dureeEnSecondes % 60 pour les secondes.

```
interface VideoProps {
  video: { id: number; titre: string; dureeEnSecondes: number };
}

function Video({ video }: VideoProps) {
  const minutes = Math.floor(video.dureeEnSecondes / 60);
```

```
  const secondes = video.dureeEnSecondes % 60;
  return (
    <div>
      <strong>{video.titre}</strong>
      <span> — {minutes} min {secondes.toString().padStart(2, '0')}</span>
    </div>
  );
}

export default Video;
```

Importez Video dans App avec au moins 2 données et affichez-les. Vérifiez dans le navigateur.

B7. Checkpoint

- Qu’est-ce qu’une prop ? Pourquoi ne doit-on pas la modifier dans l’enfant ?
- Pourquoi faut-il une clé (key) sur les éléments d’une liste ?
- Que fait {obj.propriete} dans le JSX ?

5. Étape C — Rendre l'application interactive (40 min)

C1. Objectif

Ajouter un champ de recherche qui filtre la liste, et un bouton qui trie. Ici vous découvrez la vraie nature de React : quand une valeur change (state), l'interface se recalcule.

C2. useState : la base

Modifiez src/App.tsx pour gérer une recherche :

```
import { useState } from 'react';
// ... les imports Header, CarteTechnologie, technologies

function App() {
  const [recherche, setRecherche] = useState('');

  const technologiesFiltrees = technologies.filter((tech) =>
    tech.nom.toLowerCase().includes(recherche.toLowerCase())
  );

  return (
    <main>
      <Header titre="TechTrends – Galerie des tendances" phrase="Recherchez une technologie" />
      <input
        type="search"
        placeholder="Rechercher (ex. docker)"
        value={recherche}
        onChange={(e) => setRecherche(e.target.value)}
      />
      <section>
        {technologiesFiltrees.map((tech) => (
          <CarteTechnologie key={tech.id} tech={tech} />
        ))}
      </section>
    </main>
  );
}

export default App;
```

C3. Décodez chaque ligne

Code	Signification
useState('')	Crée un état initialisé à chaîne vide. Retourne [valeur, setValeur].
value={recherche}	Le champ affiche toujours l'état (champ « contrôlé »).
onChange={(e) => setRecherche(e.target.value)}	Chaque frappe met à jour l'état.
technologiesFiltrees	Nouveau tableau calculé à partir de l'état ; le rendu s'adapte.

C4. Ajouter un second état : le tri

Ajoutez un état triCroissant (booléen) et un bouton qui l'inverse. En français :

```
const [triCroissant, setTriCroissant] = useState(true);

// copie triée (on ne mute jamais le tableau source)
const technologiesTriees = [...technologiesFiltrees].sort((a, b) =>
  triCroissant ? a.nom.localeCompare(b.nom) : b.nom.localeCompare(a.nom)
);

// dans le JSX, avant la liste :
<button onClick={() => setTriCroissant(!triCroissant)}>
  Trier ({triCroissant ? 'A → Z' : 'Z → A'})
</button>
```

Remarque : [...technologiesFiltrees] copie le tableau avant de trier. On ne modifie jamais un tableau qui vient d'un état. Les données en React sont immuables : on crée de nouvelles valeurs.

C5. Faire un composant Recherche réutilisable

Déplacez le champ de recherche dans un composant src/components/Recherche.tsx qui reçoit deux props : valeur (string) et onChange (fonction). Typez la prop onChange ainsi :

```
interface RechercheProps {
  valeur: string;
  onChange: (valeur: string) => void;
}

function Recherche({ valeur, onChange }: RechercheProps) {
  return (
    <input
      type="search"
      placeholder="Rechercher (ex. docker)"
      value={valeur}
      onChange={(e) => onChange(e.target.value)}
    />
  );
}

export default Recherche;
```

L'enfant ne possède pas l'état ; il appelle une fonction donnée par le parent. C'est le pattern « controlled component » : le parent seul décide.

C6. Exercice — le compteur officiel du développeur

Dans App, ajoutez un compteur de cartes affichées :

```
const nbCartes = technologiesTriees.length;
// affichez dans le JSX : {nbCartes} technologie(s) affichée(s)
```

Puis vérifiez que le nombre change en tapant dans le champ : la recherche et l'UI sont synchronisées parce que tout découle de l'état recherche.

C7. Checkpoint

- Pourquoi utiliser useState et pas une simple variable let tableau = … ?
- Que veut dire « champ contrôlé » ?
- Pourquoi faire une copie [...tab] avant de trier ?

6. Étape D — Afficher les données d'une API (40 min)

D1. Objectif

Remplacer les données locales par un appel réseau. Vous apprendrez le pattern le plus important du React moderne : charger des données à l'installation du composant avec useEffect, afficher un état chargement, une erreur, et les données.

D2. La mécanique en trois phases

Un appel API en React suit toujours le même squelette :

- Phase 1 — état initial : les données sont null, chargement true.
- Phase 2 — après le montage (useEffect) : on lance fetch, puis on met à jour l'état au retour.
- Phase 3 — rendu conditionnel : selon l'état (chargement / erreur / données), on affiche du JSX différent.

D3. Exemple complet — consommer une API publique

L'API JSONPlaceholder fournit de fausses données prêtes à l'emploi : https://jsonplaceholder.typicode.com/posts. Créez src/components/ListePosts.tsx :

```
import { useEffect, useState } from 'react';

interface Post {
  id: number;
  title: string;
  body: string;
}

function ListePosts() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [chargement, setChargement] = useState(true);
  const [erreur, setErreur] = useState('');

  useEffect(() => {
    let annule = false;

    async function charger() {
      try {
        const reponse = await fetch('https://jsonplaceholder.typicode.com/posts');
        if (!reponse.ok) {
          throw new Error('Réponse du serveur incorrecte : ' + reponse.status);
        }
        const donnees: Post[] = await reponse.json();
        if (!annule) {
          setPosts(donnees);
          setChargement(false);
        }
      } catch (e) {
        if (!annule) {
          setErreur(e instanceof Error ? e.message : 'Erreur inconnue');
          setChargement(false);
        }
      }
    }

    charger();
    return () => {
```

```
      annule = true;
    };
  }, []);

  if (chargement) {
    return <p>Chargement des données…</p>;
  }

  if (erreur) {
    return <p role="alert" style={{ color: '#c62828' }}>Erreur : {erreur}</p>;
  }

  return (
    <ul>
      {posts.slice(0, 10).map((post) => (
        <li key={post.id}>
          <strong>{post.title}</strong>
          <p>{post.body}</p>
        </li>
      ))}
    </ul>
  );
}

export default ListePosts;
```

D4. Décodage ligne par ligne

Élément	Rôle
useState<Post[]>([])	L'état est typé : un tableau de Post. Initialisé vide.
useEffect(() => { ... }, [])	Le tableau vide [] = l'effet ne s'exécute qu'une fois, au montage.
let annule = false	Protection anti-course : si le composant est détruit, on ignore le retour réseau.
await fetch(...)	Envoie la requête puis attend la réponse (asynchrone).
if (!reponse.ok)	Les erreurs HTTP (404, 500) ne lèvent pas d'exception : on les détecte.
setPosts(donnees)	On range les données dans l'état : React redessine la liste.
return () => { annule = true }	Fonction de nettoyage exécutée quand le composant est démonté.

D5. Rendre ListePosts visible

Importez ListePosts dans App et affichez-le sous la galerie :

```
// dans src/App.tsx, après la section existante
<section>
  <h2>Derniers articles</h2>
  <ListePosts />
</section>
```

Testez : la liste apparaît après un bref « Chargement… », puis les 10 premiers posts. Ouvrez l'onglet Réseau de DevTools (F12) : la requête /posts doit s'afficher avec le statut 200.

D6. Exercice — adapter l'exemple à votre galerie

Remplacez les données locales par un appel à l'API suivante qui fournit des posts avec vos ids : https://jsonplaceholder.typicode.com/users/1/todos (liste de tâches).

- Créez une interface Todo { id: number; title: string; completed: boolean } dans un fichier src/types.ts.
- Créez src/components/ListeTaches.tsx qui charge ces todos et les affiche avec une coche ✅/☐ pour completed.
- Gérez les trois états : chargement, erreur, données.
- Réutilisez la fonction de nettoyage annule vue en D3.

Si vous écrivez ce composant sans regarder D3, vous maîtrisez le pattern. Sinon relisez D3 puis recommencez : la clé, c'est de savoir l'écrire de tête.

D7. (Recommandé) Consommer VOTRE API du TP3

Si votre API Express du TP3 (Produits) est encore en route à http://localhost:3000, créez src/components/ListeProduits.tsx qui réutilise le même pattern avec votre endpoint. Pensez au CORS : si les navigateurs bloquent la requête, ajoutez le middleware cors à votre Express :

```
npm install cors

import cors from 'cors';
app.use(cors());
```

D8. Checkpoint

- Pourquoi le tableau [] en deuxième argument de useEffect ? Que se passerait-il sans tableau ?
- Pourquoi ne peut-on pas rendre l'interface avant que les données arrivent ?
- Quelle différence entre un try/catch et le if (!reponse.ok) ?

7. Étape E (bonus) — Formulaire et déploiement (30 min)

E1. Un formulaire contrôlé

Composez un formulaire « ajouter une technologie » dont chaque champ est contrôlé par un état :

```
import { useState } from 'react';

function AjouterTechnologie() {
  const [nom, setNom] = useState('');
  const [message, setMessage] = useState('');

  function soumettre(e: React.FormEvent) {
    e.preventDefault(); // ne pas recharger la page
    setMessage('« ' + nom + ' » envoyé !');
    setNom('');
  }

  return (
    <form onSubmit={soumettre}>
      <input
        value={nom}
        onChange={(e) => setNom(e.target.value)}
        placeholder="Nom de la technologie"
      />
      <button type="submit">Ajouter</button>
      <p>{message}</p>
    </form>
  );
}

export default AjouterTechnologie;
```

Remarque : e.preventDefault() est OBLIGATOIRE, sinon la page se recharge et perd l'état.

E2. Préparer le build de production

```
npm run build
```

Cette commande crée le dossier dist/ : du HTML, du CSS et du JavaScript optimisés et minifiés. Pour vérifier en local comme en production :

```
npm run preview
```

E3. Déployer sur Vercel (recommandé)

Vercel est l'hébergeur officiel recommandé pour React. Deux façons :

- Depuis le site vercel.com : Import Project → connecter le dépôt GitHub → Vercel détecte automatiquement Vite (build : npm run build, output : dist) → Deploy.
- En ligne de commande : npm i -g vercel puis vercel (l'outil vous guide), puis vercel --prod.

Résultat : vous obtenez une adresse publique du type https://techtrends-react.vercel.app. C'est cette URL que vous remettrez !

E4. Déployer sur Render

Alternative : Render.com, qui accepte les sites statiques. Import du dépôt GitHub → type "Static Site" → Build command : npm run build → Publish directory : dist. Le premier déploiement prend quelques minutes.

Erreur fréquente au déploiement

Si Vercel affiche une erreur « No Output Directory named dist », vérifiez que le champ Build Command est npm run build et Output Directory est dist.

Si la page blanche apparaît : ouvrez la console du navigateur ; en général c'est une erreur JS dans la console, lisez-la.

E5. Checklist de remise

Étape	Vu ?
npm run dev fonctionne en local	
La galerie filtre, trie, affiche le compteur	
Liste posts / taches charge depuis l'API avec gestion d'erreur	
npm run build passe sans erreur	
Le site est en ligne (lien Vercel ou Render)	
Le code est sur GitHub (public) et le lien est fourni	

8. Casser pour comprendre

Volontairement, cassez le code pour observer les messages d'erreur. C'est le meilleur exercice pour apprendre à lire ce que TypeScript et React vous disent.

- Retirez les accolades autour de {tech.nom} dans CarteTechnologie : que se passe-t-il ? Pourquoi ?
- Renvoyez deux éléments frères dans App sans les envelopper dans un : que lève TypeScript ? (c'est la règle « un seul élément racine »).
- Typez une prop avec la mauvaise valeur : ex. titre={42} dans Header. Lisez le message complet de TS.
- Oubliez key sur la liste : regardez l'avertissement dans la console du navigateur (Warning: Each child...).
- Mettez une URL fetch fausse : vérifiez que votre gestion d'erreur s'affiche (et pas un écran blanc).

9. Bilan, questions et remise

9.1 Questions de compréhension (à rendre)

Répondez dans un fichier REPONSES.md à la racine du projet, 3 à 5 lignes par question :

- Expliquez ce qu'est React et en quoi il diffère de JavaScript « vanilla » qui manipule le DOM à la main.
- Expliquez le rôle du DOM virtuel : que fait React quand une donnée change ?
- Quelle est la différence entre une prop et un state ? Donnez un exemple des deux.
- Pourquoi écrit-on des fichiers .tsx et pas seulement .ts ?
- Que fait le tableau [] en deuxième argument de useEffect, et pourquoi la liste des tasks doit l'utiliser ?
- Expliquez comment vous gérez l'état de chargement, d'erreur et de données quand vous appelez une API.
- Quels sont les 3 dossiers couches d'un projet Vite React que vous diriez à un débutant ?

9.2 Ce que vous remettez

Élément	Format
Lien GitHub du code source (repo public)	URL
Lien du site déployé (Vercel ou Render)	URL
REPONSES.md	Fichier Markdown
Contrôle oral (2 min)	Explications sur votre code

9.3 Grille de correction (/100)

Critère	Pondération
Project créé avec Vite + TypeScript, structure propre, npm run dev OK	15
Composants découpés (Header, Carte, Recherche, Liste) et réutilisés	20
Props typées correctement (interfaces) dans au moins 2 composants	15

État et événements : recherche + tri fonctionnels, champ contrôlé	20
Chargement d'API : gestion chargement / erreur / données, nettoyage	15
Déploiement en ligne et lien valide	10
REPONSES.md complet et exact	5
Soustraction : code non compris au contrôle oral	- jusqu'à -50

9.4 Ce que je regarde en premier au contrôle oral

- « Montre-moi ta prop : pourquoi n'as-tu pas besoin de la modifier ici ? »
- « Que contient ton state ? Quand React redessine-t-il ? »
- « Pourquoi ce tableau [] ? Que lancerais-tu si une donnée changeait ? »
- « Ton composant gère-t-il une erreur réseau ? Montre-moi. »
- « Explique ta ligne fetch ».

Annexe A — Erreurs fréquentes

Erreur / Symptôme	Cause probable	Correctif
Page blanche, console : "Element type is invalid"	Mauvais import/export de composant	Vérifier default vs nommé, et le chemin
Cannot read properties of undefined (reading "nom")	on tente d'afficher une donnée avant son chargement	Afficher les données seulement si présent (phase 3)
TS2322: Type 'number' is not assignable to type 'string'	Prop mal typée	Corriger l'interface ou la valeur
Input qui ne tape pas / réapparaît	Champ non contrôlé ou valeur non liée	value={état} + onChange={set}
Le useEffect tourne en boucle infinie	Mettre fonctions/tableaux dans les dépendances ou oublier []	Dépendances correctes (ici [])
CORS error sur la console	API externe sans head erg CORS	cors sur Express (voir D7)
"No Output Directory named dist" sur Vercel	Mauvaise config build	Build = npm run build, output = dist

Annexe B — Commandes utiles

```
# création
npm create vite@latest mon-app -- --template react-ts

# installation / dev / build / préview
npm install
npm run dev
npm run build
npm run preview

# déploiement Vercel (option ligne de commande)
npm i -g vercel
vercel
vercel --prod
```

Annexe C — Links utiles

- Documentation React (français) : https://fr.react.dev/
- Guide Vite : https://vitejs.dev/guide/
- Template React + TS : documenté dans votre projet (README généré).
- JSONPlaceholder (API de test) : https://jsonplaceholder.typicode.com/

Annexe D — Le quotidien WebStorm

Action	Raccourci WebStorm
Ouvrir Terminal intégré	Alt+F12
Lancer le npm script (dev/build)	L'onglet « npm » en haut à gauche ; double-clic sur run dev

Reformater le code	Ctrl+Alt+L
Renommer un symbole partout	Maj+F6
Rechercher un fichier	Double Maj
Déplacer la souris dans un warning	Alt+Entrée (Intention)